Universidad Autónoma de Campeche

Facultad de Ingenieria

Concurrencia y Paralelismo

Tecnologías de Software

5to “A”

PRACTICA 03

Servidor Concurrente

2do parcial

29 de Octubre de 2026

Alumno: Victor Adrian Romero Minaya

Matricula: 67350

Docente: Sergio A Noh Puch

1. Mecanismo de Salida Limpia

La palabra clave para terminar la conexión es “salir” asi en minusculas.

En el cliente:

- El usuario escribe salir y el cliente lo envía al servidor.
- La condición del ciclo while (!mensaje.equals("salir")) deja de cumplirse y el ciclo termina.
- Se imprime "Sesión terminada." y el programa llega al final del bloque try, donde Java cierra el socket automáticamente y yap.

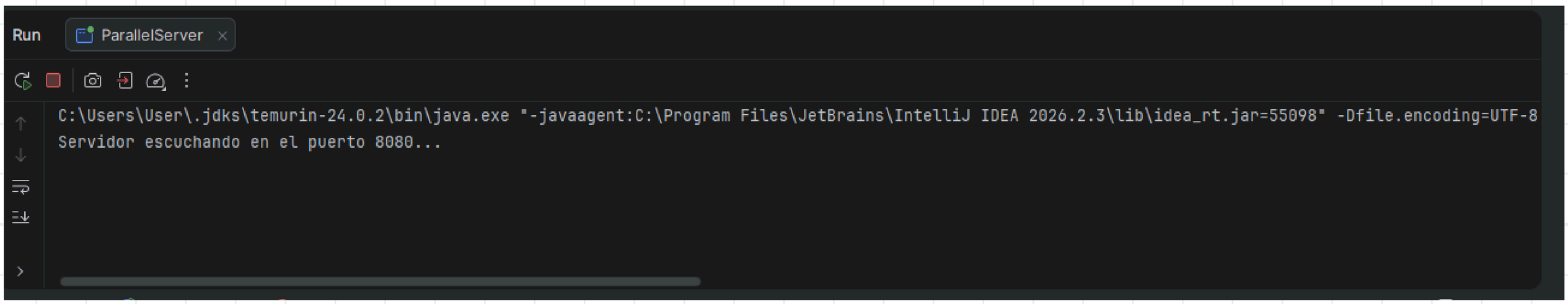
En el servidor:

- Recibe el mensaje salir, responde "Adiós, cerrando conexión" asi bien frio y ejecuta break para salir del ciclo.
- Se imprime "Cliente desconectado." y se llega al final del bloque try, donde Java cierra los streams (in, out) y el socket de ese cliente.
- El hilo del pool que atendía a ese cliente termina handleRequest y queda libre para otro cliente.

Como los recursos se declaran dentro de los paréntesis del try, se cierran solos aunque ocurra un error, así que no quedan sockets abiertos ni hilos atorados. El servidor sigue encendido esperando nuevos clientes.

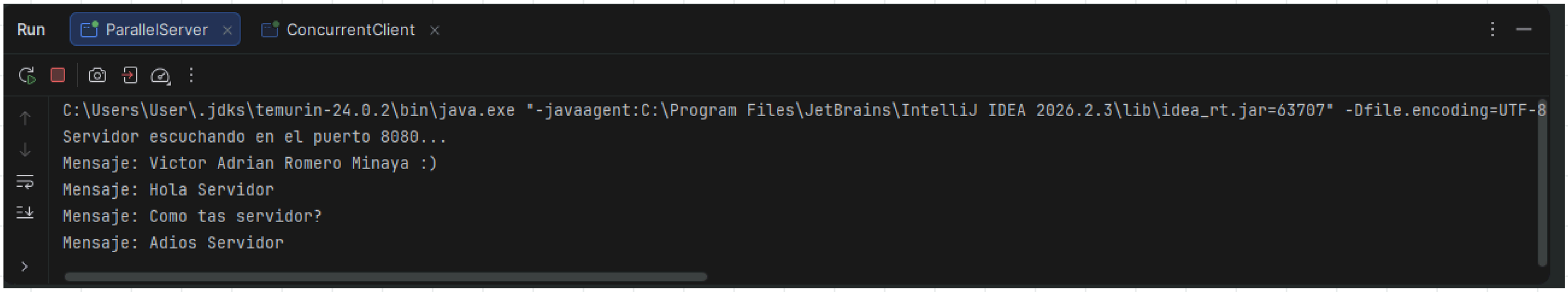
2. Capturas

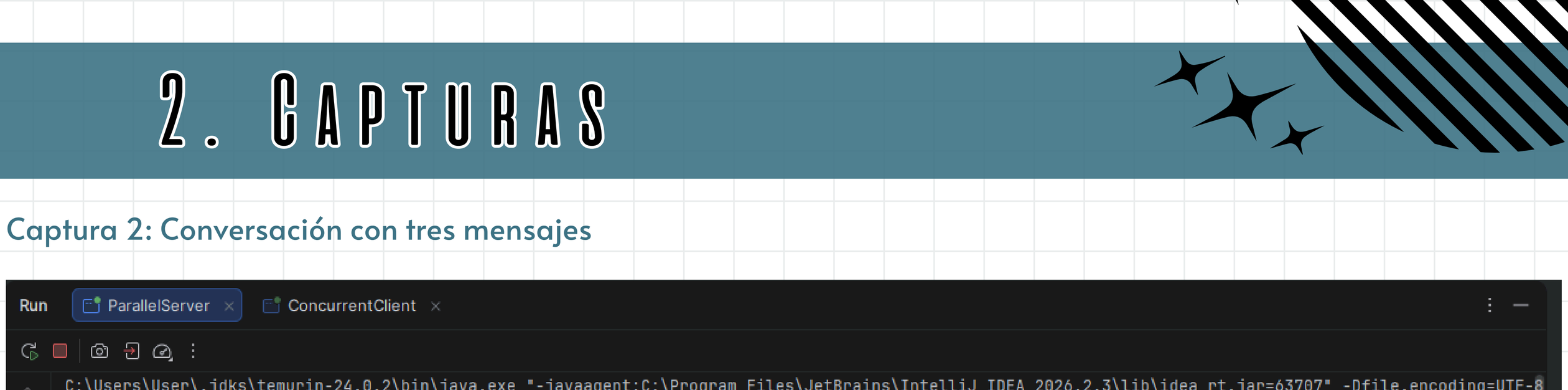
Captura 1: Servidor iniciado



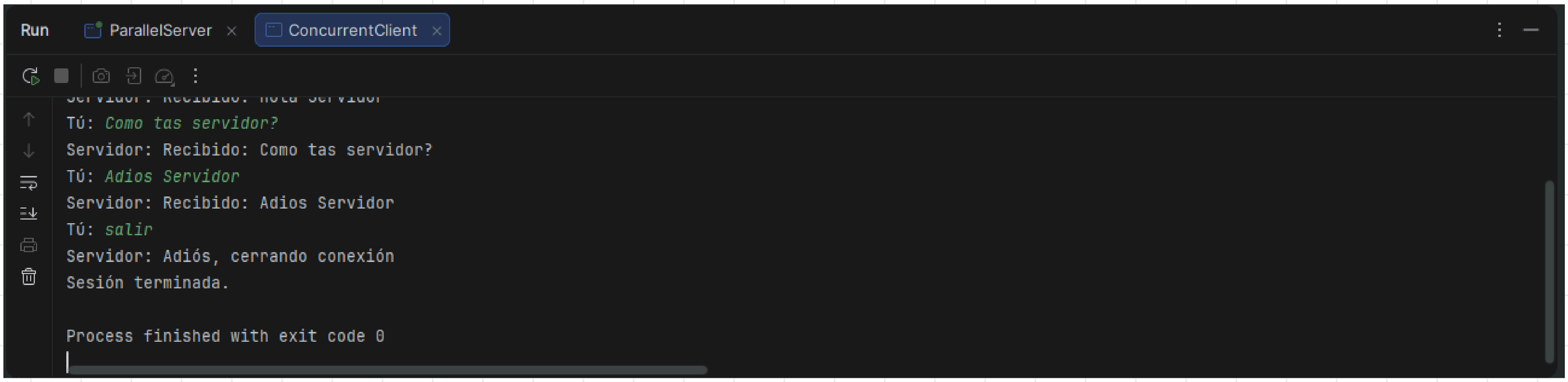
2. Capturas

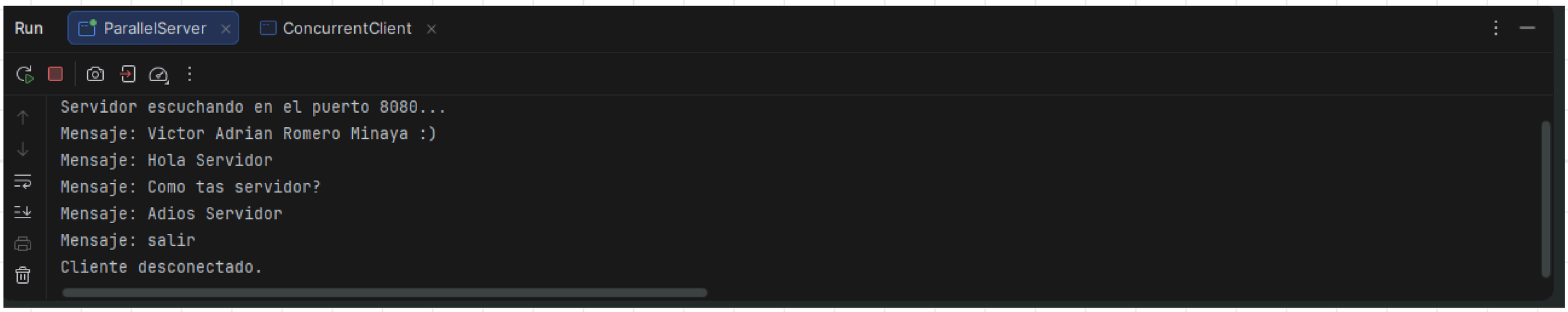
Captura 2: Conversación con tres mensajes





Captura 2: Conversación con tres mensajes





3- Explicación

Qué cambió respecto al código original: En el código anterior que tenia el cliente enviaba un solo mensaje y terminaba el programa, y el servidor leía una sola línea con readLine() y salía del método, lo que cerraba el socket. Ahora ambos tienen un ciclo while: el cliente lee lo que escribo, lo envía y espera la respuesta; el servidor lee cada mensaje, lo imprime y responde.

Por qué la conexión se mantiene viva: El socket solo se cierra cuando el programa llega al final del bloque try. Mientras el ciclo siga activo, eso no ocurre. El servidor se queda en readLine() esperando el siguiente mensaje sin cerrar nada, y el ciclo solo termina cuando el usuario escribe "salir", ya es toda we.

Qué pasa cuando el cliente deja de enviar datos: Si el cliente simplemente no escribe nada, el servidor se queda bloqueado en readLine() esperando, y la conexión sigue abierta. Si el cliente se cierra o se desconecta, readLine() devuelve null, el ciclo while (msj != null) termina, se imprime "Cliente desconectado." y se cierran los recursos, liberando el hilo del pool.



¡Gracias por leer!